AGENT 1: Foundation, Barcode Engine, and Data Input (Phases 1–3)

You are building a full-stack GS1 DataMatrix UDI Label Management System for medical devices. This system must comply with FDA 21 CFR 801.45 and EU MDR 2017/745 Article 27. You are responsible for Phases 1–3: project foundation, barcode engine, and data input with live preview.

Read MASTER-INSTRUCTIONS.md first for the complete product catalog and architecture decisions.

CRITICAL LABEL FORMAT RULE

LOT MODE: (01) GTIN + (10) Lot + (11) Manufacturing. NO expiration. HRI: GTIN → Lot → MfgDate (3 lines). **SERIAL MODE:** (01) GTIN + (17) Expiration + (11) Manufacturing + (21) Serial. WITH expiration. HRI: GTIN → Serial → MfgDate → ExpDate (4 lines).

DataMatrix ENCODING order (fixed-length AIs first):

- LOT: `(01)GTIN(11)YYMMDD(10)LOT`
- SERIAL: `(01)GTIN(17)YYMMDD(11)YYMMDD(21)SERIAL`

HRI DISPLAY order (identifier after GTIN):

- LOT: `(01) GTIN` → `(10) LOT` → `(11) YYMMDD`
- SERIAL: `(01) GTIN` → `(21) SERIAL` → `(11) YYMMDD` → `(17) YYMMDD`

TECH STACK (Do not substitute)

- Frontend: React 18+ with TypeScript, Vite, Tailwind CSS
- Backend: Node.js with Express and TypeScript
- Storage: JSON files on disk (NO SQL database — max 6 users)
- Barcode: bwip-js (gs1datamatrix symbology)
- Date math: date-fns
- Validation: Zod (backend), react-hook-form + zod resolver (frontend)
- Auth: jsonwebtoken, bcryptjs, HTTP-only cookies
- Concurrency-safe file I/O: proper-lockfile

DATA STORAGE: JSON FILES

Storage Architecture

```
/data/
├── users.json
├── products.json          # 39 pre-seeded products
├── templates.json
├── printers.json
├── print-jobs.json
├── serial-counters.json    # NEW: tracks last-used serial per prefix family per year
└── audit-log.json
```

JSON Store Service

Create `server/src/services/jsonStore.ts` — a generic, reusable, concurrency-safe JSON file store:

```typescript
```

```
import fs from 'fs/promises';
import path from 'path';
import lockfile from 'proper-lockfile';

const DATA_DIR = path.join(process.cwd(), 'data');

export class JsonStore<T extends { id: string }> {
  private filePath: string;

  constructor(filename: string) {
    this.filePath = path.join(DATA_DIR, filename);
  }

  async init(): Promise<void> {
    await fs.mkdir(DATA_DIR, { recursive: true });
    try {
      await fs.access(this.filePath);
    } catch {
      await fs.writeFile(this.filePath, '[]', 'utf-8');
    }
  }

  async readAll(): Promise<T[]> {
    const release = await lockfile.lock(this.filePath, { retries: 5 });
    try {
      const raw = await fs.readFile(this.filePath, 'utf-8');
      return JSON.parse(raw) as T[];
    } finally {
      await release();
    }
  }

  async write(data: T[]): Promise<void> {
    const release = await lockfile.lock(this.filePath, { retries: 5 });
    try {
      await fs.writeFile(this.filePath, JSON.stringify(data, null, 2), 'utf-8');
    } finally {
      await release();
    }
  }

  async append(item: T): Promise<void> {
    const all = await this.readAll();
```

```
    all.push(item);
    await this.write(all);
  }

  async findById(id: string): Promise<T | undefined> {
    const all = await this.readAll();
    return all.find(item => item.id === id);
  }

  async updateById(id: string, partial: Partial<T>): Promise<T | null> {
    const all = await this.readAll();
    const index = all.findIndex(item => item.id === id);
    if (index === -1) return null;
    all[index] = { ...all[index], ...partial, id };
    await this.write(all);
    return all[index];
  }

  async deleteById(id: string): Promise<boolean> {
    const all = await this.readAll();
    const filtered = all.filter(item => item.id !== id);
    if (filtered.length === all.length) return false;
    await this.write(filtered);
    return true;
  }
}
```

Create `server/src/services/auditLogStore.ts` — append-only (no update/delete/overwrite). IDs are auto-incrementing integers.

Create `server/src/stores/index.ts`:

```typescript
```

```
export const userStore = new JsonStore<User>('users.json');
export const productStore = new JsonStore<Product>('products.json');
export const templateStore = new JsonStore<LabelTemplate>('templates.json');
export const printerStore = new JsonStore<Printer>('printers.json');
export const printJobStore = new JsonStore<PrintJob>('print-jobs.json');
export const serialCounterStore = new JsonStore<SerialCounter>('serial-counters.json');
export const auditLogStore = new AuditLogStore('audit-log.json');

export async function initializeStores(): Promise<void> {
  // Init all stores, seed admin user, seed 39 products
}
```

Data Schemas

users.json — same as original spec (see MASTER-INSTRUCTIONS.md)

products.json — 39 pre-seeded products:

```json
{
  "id": "uuid",
  "name": "PV1010-23CI - Maxxi Gold Electrode 96\"",
  "gtin": "00850008393006",
  "partNumber": "PV1010-23CI",
  "description": "",
  "identifierMode": "lot",
  "lotPrefix": "MG96CI",
  "serialPrefixFamily": null,
  "shelfLifeYears": 0,
  "shelfLifeMonths": 0,
  "shelfLifeDays": 0,
  "autoExpirationEnabled": false,
  "createdBy": "system",
  "createdAt": "2026-02-18T00:00:00Z",
  "updatedAt": "2026-02-18T00:00:00Z"
}
```

serial-counters.json — NEW:

```json
```

```
{
  "id": "bwiii-2026",
  "prefixFamily": "BWIII",
  "year": 2026,
  "lastUsedNumber": 3047,
  "updatedAt": "2026-02-18T10:30:00Z"
}
```

print-jobs.json — includes serial range for audit:

```json
{
  "id": "uuid",
  "userId": "user-uuid",
  "templateId": "template-uuid",
  "printerId": "printer-uuid",
  "productId": "product-uuid",
  "identifierMode": "serial",
  "gtin": "00850008393211",
  "lotNumber": null,
  "serialPrefixFamily": "BWIII",
  "serialYear": 2026,
  "serialRangeStart": 3000,
  "serialRangeEnd": 3049,
  "manufacturingDate": "2026-02-18",
  "expirationDate": "2033-02-18",
  "totalLabels": 50,
  "copiesPerLabel": 1,
  "status": "completed",
  "labelsPrinted": 50,
  "errorMessage": null,
  "createdAt": "2026-02-18T10:30:00Z",
  "completedAt": "2026-02-18T10:31:15Z"
}
```

audit-log.json and **templates.json** and **printers.json** — same as original spec.

PROJECT STRUCTURE

```
/
```

```
├── client/
│   ├── src/
│   │   ├── components/
│   │   │   ├── Auth/
│   │   │   │   ├── LoginPage.tsx
│   │   │   │   └── UserManagement.tsx
│   │   │   ├── LabelForm/
│   │   │   │   ├── LabelDataForm.tsx
│   │   │   │   ├── ProductSelector.tsx       # NEW: replaces manual GTIN + mode toggle
│   │   │   │   ├── GtinDisplay.tsx           # NEW: read-only GTIN from product
│   │   │   │   ├── LotInput.tsx              # NEW: auto-generated lot with override
│   │   │   │   ├── SerialInput.tsx           # NEW: auto-suggested serial with edit
│   │   │   │   ├── DateInputs.tsx
│   │   │   │   └── ExpirationCalculator.tsx
│   │   │   ├── LabelPreview/
│   │   │   │   ├── LabelPreview.tsx          # NEW: layout engine (stacked vs side-by-side)
│   │   │   │   ├── DataMatrixPreview.tsx
│   │   │   │   ├── HriTextPreview.tsx
│   │   │   │   └── PreviewControls.tsx       # NEW: size slider + alignment buttons
│   │   │   ├── ProductManager/
│   │   │   │   ├── ProductList.tsx
│   │   │   │   └── ProductForm.tsx
│   │   │   └── Layout/
│   │   │       ├── AppShell.tsx
│   │   │       ├── Sidebar.tsx
│   │   │       └── ProtectedRoute.tsx
│   │   ├── services/
│   │   │   ├── api.ts
│   │   │   └── barcodePreview.ts
│   │   ├── hooks/
│   │   │   ├── useAuth.ts
│   │   │   └── useProducts.ts
│   │   ├── types/
│   │   │   ├── gs1.types.ts
│   │   │   ├── auth.types.ts
│   │   │   └── product.types.ts
│   │   ├── utils/
│   │   │   ├── gtinValidator.ts
│   │   │   ├── gs1Encoder.ts
│   │   │   ├── serialGenerator.ts
│   │   │   └── labelLayout.ts              # NEW: computeLayout() for preview sizing
│   │   ├── App.tsx
│   │   └── main.tsx
│   ├── index.html
```

```
│   ├── tailwind.config.js
│   ├── tsconfig.json
│   ├── vite.config.ts
│   └── package.json
├── server/
│   ├── src/
│   │   ├── routes/
│   │   │   ├── auth.routes.ts
│   │   │   ├── products.routes.ts
│   │   │   ├── barcode.routes.ts
│   │   │   └── serialCounter.routes.ts     # NEW
│   │   ├── controllers/
│   │   │   ├── auth.controller.ts
│   │   │   ├── products.controller.ts
│   │   │   └── serialCounter.controller.ts  # NEW
│   │   ├── services/
│   │   │   ├── jsonStore.ts
│   │   │   ├── auditLogStore.ts
│   │   │   ├── barcode.service.ts
│   │   │   └── audit.service.ts
│   │   ├── middleware/
│   │   │   ├── auth.middleware.ts
│   │   │   ├── rbac.middleware.ts
│   │   │   └── validation.middleware.ts
│   │   ├── stores/
│   │   │   └── index.ts
│   │   └── app.ts
│   ├── tsconfig.json
│   └── package.json
├── data/
│   └── .gitkeep
├── .env.example
├── .gitignore
└── README.md
```

PHASE 1: PROJECT FOUNDATION

Step 1.1 — Initialize the monorepo

Root `package.json`:

```
json
```

```
{
  "scripts": {
    "dev": "concurrently \"cd server && npm run dev\" \"cd client && npm run dev\"",
    "build": "cd client && npm run build && cd ../server && npm run build",
    "start": "cd server && node dist/app.js",
    "setup": "npm install && cd client && npm install && cd ../server && npm install"
  }
}
```

Client dependencies:

```bash
npm install tailwindcss @tailwindcss/vite react-router-dom react-hook-form @hookform/resolvers zod axios date-fns bwip-js
npm install -D @types/bwip-js
```

Server dependencies:

```bash
npm install express cors helmet cookie-parser jsonwebtoken bcryptjs zod dotenv uuid proper-lockfile
npm install -D typescript @types/express @types/cors @types/cookie-parser @types/jsonwebtoken @types/bcryptjs @types/
```

Configure Vite to proxy `/api` to `http://localhost:3001`.

Step 1.2 — JSON Store + Stores

Implement JsonStore, AuditLogStore, and all store instances as described above.

Step 1.3 — Seed Data

On first run (when files are empty):

1. **Admin user:** username=`admin`, password=`ChangeMe123!` (bcrypt cost 12), role=`admin`
2. **All 39 products** from the catalog in MASTER-INSTRUCTIONS.md
3. **Serial counters:** initialize `BWIII-{currentYear}` and `BWM-{currentYear}` both at `lastUsedNumber: 2999` (so first use starts at 3000)
4. **Default templates** (created in Phase 4, but seed empty array for now)

Step 1.4 — Authentication System

Backend:

- `POST /api/auth/login` — validate against userStore, issue JWT in HTTP-only cookie `auth_token`
- `POST /api/auth/logout` — clear cookie
- `GET /api/auth/me` — return current user
- `POST /api/auth/users` — admin only, max 6 users
- `GET /api/auth/users` — admin only
- `PATCH /api/auth/users/:id` — admin only: update role, deactivate, reset password
- All mutations log to audit trail

Frontend:

- `useAuth` hook with context: user, login(), logout(), isAuthenticated, isAdmin
- LoginPage, ProtectedRoute, AppShell with Sidebar, UserManagement

Step 1.5 — .env and .gitignore

`.env.example`:

```
JWT_SECRET=change-this-to-a-random-64-char-string
JWT_EXPIRY=30m
PORT=3001
NODE_ENV=development
```

Step 1.6 — Test Phase 1

1. `npm run dev` starts both client and server
2. `/data` directory created with seeded JSON files
3. `products.json` contains all 39 products
4. `serial-counters.json` has BWIII and BWM entries
5. Login with admin/ChangeMe123! works
6. Create operator user → saved to users.json
7. 7th user rejected
8. Audit log has entries

PHASE 2: BARCODE ENGINE & GS1 ENCODING

Step 2.1 — GTIN-14 Validator

client/src/utils/gtinValidator.ts:

```typescript
export function validateGtin14(gtin: string): { valid: boolean; error?: string }
export function computeGtinCheckDigit(first13: string): number
```

Step 2.2 — GS1 Element String Encoder

client/src/utils/gs1Encoder.ts:

```typescript
export interface GS1LabelData {
  gtin: string;
  identifierMode: 'lot' | 'serial';
  lotNumber?: string;
  serialNumber?: string;
  manufacturingDate: Date;
  expirationDate?: Date;  // ONLY present in serial mode
}

// ENCODING ORDER (for DataMatrix barcode):
// LOT: "(01)GTIN(11)YYMMDD(10)LOTVALUE"
// SERIAL: "(01)GTIN(17)YYMMDD(11)YYMMDD(21)SERIALVALUE"
export function buildGS1ElementString(data: GS1LabelData): string

// HRI DISPLAY ORDER (for human-readable text on label):
// LOT: ["(01) GTIN", "(10) LOT", "(11) YYMMDD"] — 3 lines
// SERIAL: ["(01) GTIN", "(21) SERIAL", "(11) YYMMDD", "(17) YYMMDD"] — 4 lines
export function buildHriText(data: GS1LabelData): string[]

export function formatGS1Date(date: Date): string // → YYMMDD
```

Step 2.3 — Serial Number Generator

client/src/utils/serialGenerator.ts:

```typescript
```

```typescript
export interface SerialPattern {
  prefix: string;    // e.g., "BWIII2026-"
  suffix: string;
  start: number;     // e.g., 3000
  end: number;       // e.g., 3049
  step: number;
  padWidth: number;
}
export function generateSerials(pattern: SerialPattern): string[]
export function countSerials(pattern: SerialPattern): number
```

Step 2.4 — Auto Lot Number Generator

`client/src/utils/lotGenerator.ts`:

```typescript
// Given a lot prefix and manufacturing date, generate the lot number
// e.g., lotPrefix="MG96CI", mfgDate=2026-02-18 → "MG96CI-260218"
export function generateLotNumber(lotPrefix: string, mfgDate: Date): string
```

Step 2.5 — Server-side Barcode Service

`server/src/services/barcode.service.ts`:

- `generateDataMatrixPng(gs1ElementString, moduleSize)` using bwip-js `gs1datamatrix`
- Route: `POST /api/barcode/preview` — returns PNG

Step 2.6 — Serial Counter API

`server/src/routes/serialCounter.routes.ts`:

- `GET /api/serial-counters/:prefixFamily/:year` — get last used number
- `PATCH /api/serial-counters/:prefixFamily/:year` — update after batch print completes
- Auto-creates counter entry if it doesn't exist for a new year (starting at 2999)

Step 2.7 — Test Phase 2

1. All GTIN-14s from the 39 products validate correctly
2. LOT mode encoding: NO AI 17 anywhere
3. SERIAL mode encoding: AI 17 present

4. HRI line 2 is always the identifier (lot or serial), not a date
5. Auto lot: `MG96CI` + `2026-02-18` → `MG96CI-260218`
6. Serial counter: read → 2999, print 50 → update to 3049, next read → 3049

PHASE 3: LABEL DATA INPUT & PREVIEW UI

Step 3.1 — Label Data Form

Key change from original spec: The user selects a PRODUCT, not a mode. The product determines the mode.

`ProductSelector.tsx`:

- Dropdown with products grouped by mode:
  - `<optgroup label="Serial Mode (with expiration)">` — 9 products
  - `<optgroup label="Lot Mode (no expiration)">` — 30 products
- Display format in dropdown: product name (which is `{Part} - {Device}`)
- Below dropdown: show mode badge, GTIN (read-only), validation checkmark

`GtinDisplay.tsx`:

- Read-only display of GTIN from selected product
- Green checkmark (all pre-seeded GTINs are valid)
- GTIN is NOT editable — it comes from the product

`LotInput.tsx` (shown when product.identifierMode === 'lot'):

- Auto-generated lot number: `{product.lotPrefix}-{YYMMDD from mfg date}`
- Editable — user can override
- "Reset to auto" link when overridden

`SerialInput.tsx` (shown when product.identifierMode === 'serial'):

- Single mode: one serial number input, auto-filled from counter
- Batch mode toggle
- In batch mode: prefix (read-only, from product), start (editable, auto-suggested from counter+1), end (editable)
- Preview of serial range below

`DateInputs.tsx`:

- Manufacturing date (always shown, defaults to today)
- Expiration date (ONLY for serial mode)
  - Auto-calc toggle (default on), uses product.shelfLifeYears (always 7)
  - Manual override available

`ExpirationCalculator.tsx`:

- Only visible in serial mode
- Shows "Auto: +{N} years from manufacturing" when auto is on

Step 3.2 — Live Preview

Layout Engine (`client/src/utils/labelLayout.ts`):

```typescript
// Determines stacked vs side-by-side layout, DataMatrix size, font size
// based on label dimensions and user scale setting
export function computeLayout(widthMm: number, heightMm: number, scale: number): {
  dmMm: number;      // DataMatrix size in mm
  fontMm: number;    // HRI font size in mm
  stacked: boolean;  // true = DM on top, text below; false = side by side
}
```

Rules:

- Square labels (w ≈ h) → stacked layout
- Portrait labels (h > w) → stacked
- Wide labels where text fits beside DM → side-by-side
- Small labels (< 30mm either dimension) → stacked

PreviewControls.tsx:

- Size slider: 50% to 200% (default 100%)
- Text alignment: left / center / right buttons

DataMatrixPreview.tsx:

- bwip-js client-side canvas rendering, debounced 300ms

HriTextPreview.tsx:

- LOT: 3 lines, NO expiration
- SERIAL: 4 lines, WITH expiration
- Respects text alignment setting
- Monospace font, YYMMDD date format

Step 3.3 — Product Profiles

Products are pre-seeded. The Products page is read-only for operators, editable for admin.

API routes for `productStore`:

- `GET /api/products` — list all
- `GET /api/products/:id` — single product
- `POST /api/products` — admin only: add product
- `PATCH /api/products/:id` — admin only: update
- Do NOT enforce unique GTIN (Maxxi Position shares one)

Step 3.4 — Routing

```
/login          → LoginPage
/               → redirect to /labels
/labels         → LabelDataForm (with product selector + preview)
/products       → ProductList
/admin/users    → UserManagement (admin only)
```

Step 3.5 — Test Phase 3

1. Select a LOT product → lot input shown, auto-generated lot number, NO expiration fields
2. Select a SERIAL product → serial input shown, auto-suggested serial, expiration fields visible
3. Change manufacturing date on LOT product → lot number auto-updates
4. SERIAL batch mode → start/end both editable, preview shows range
5. LOT preview: 3 HRI lines, no (17)
6. SERIAL preview: 4 HRI lines, with (17)
7. Square 1"×1" template: stacked layout, readable text
8. Standard 2"×1" template: side-by-side layout
9. Size slider adjusts DataMatrix and font proportionally

10. Text alignment changes reflected in preview
11. Scan preview DataMatrix → LOT has no expiration, SERIAL has expiration